

MAE-A02: Coursework Submission Electronic Coversheet

Coursework **MUST** be submitted online via WISEflow unless you are told otherwise by your Module Leader.

Student Number:	20389787
Module Code:	ME5668-ME5688
Module Title:	Advanced Vehicle Dynamics & Advanced CAD
Module Tutor:	Prof Bertrand Delpch, Prof Xinli Du
Assessment/Report Title:	MATLAB Modelling Coursework

I confirm that I understand a complete submission of coursework is by one electronic copy of my assignment via WISEflow. I understand that assignments must be submitted by the deadline in order to achieve an uncapped grade. Separate guidelines apply to reassessed work. Please see the [Coursework Submission Policy](#) for details.

Any coursework or examined submission for assessment where plagiarism, collusion or any form of cheating is suspected will be dealt with according to the University processes which are detailed in [Senate Regulation 6](#).

You can access information about plagiarism [here](#).

The University's regulations on plagiarism apply to published and unpublished work, as well as collusion and plagiarism of the work of other students.

Please ensure that you fully understand what constitutes plagiarism before you submit your work.

I confirm that I have read and understood the guidance on plagiarism. I also confirm that I have neither plagiarised in this coursework nor allowed my own work to be plagiarised.

I confirm that I have read and understood the guidance on using generative artificial intelligence (AI) in this assignment. I also confirm that where the use of AI is allowed, I have appropriately acknowledged any use of AI.

The submission of this coversheet is confirmation that you have read and understood the above statements.

Part 1)

Question 1)

To analyse the vehicle's trajectory, the raw data for Yaw Rate ($\dot{\psi}$) and Vehicle Speed (V) were processed to derive the Yaw Angle (ψ) and Distance Travelled (s) with time. The MATLAB code shown in Figure 1 in the appendix shows how the raw data provided rate based values. The critical data pre-processing occurs between Lines 9 and 17 of the code. Since Simulink requires time-series inputs to be in specific formats, Line 9 forces the time vector t into a column format using the $(:)$ operator. Similarly, Line 12 converts the yaw rate to a column, and Line 13 concatenates it with time to create the matrix required for the simulation. A physical conversion happens on Line 16, where the vehicle speed is converted from km/h to m/s (dividing by 3.6) to ensure the integration calculated distance in meters. Numerical integration was required to obtain the position based values. The mathematical relationships are defined as:

$$\psi(t) = \int_0^t \dot{\psi}(t) dt \quad \text{and} \quad s(t) = \int_0^t V(t) dt$$

The Simulink model shown in Figure 2 in the appendix shows how the integration was modelled. The Simulink model utilizes two continuous-time integrator blocks to perform the numerical integration required for the analysis. The top branch integrates the yaw rate to compute the cumulative yaw angle, while the bottom branch integrates the speed to derive the total distance travelled, with both outputs exported back to the MATLAB workspace for plotting. A Fixed-Step Solver with a step size of 0.01s was selected for the simulation configuration. This specific setting was chosen to match the sampling frequency of the raw data, ensuring that the integration step aligned perfectly with the data points without introducing interpolation errors. The simulation itself is triggered on Line 20 using the `sim` command, which executes the external Simulink model "v_dynamics_model". Following the run, Lines 23–25 extract the calculated results from the simulation output object into local variables. Finally, Lines 28–46 generate the visualization.

Figure 3 shows the yaw angle and distance travelled against time which describe the vehicle's directional behaviour and overall progress along the rally stage. The yaw angle's cumulative behaviour reflects how the car's heading changes over time. In the plot, the yaw angle varies smoothly with several monotonic sections and step-like changes, which correspond to consecutive corners where the vehicle experiences sustained rotational motion. Flatter regions indicate short straights with low yaw rate. The overall continuous change in yaw without frequent sign reversals suggests that the stage predominantly consists of turns in one direction, typical of a winding rally road rather than a closed circuit. The distance–time curve increases monotonically because it represents the integral of vehicle speed; its slope equals instantaneous speed, so

steeper segments indicate acceleration or higher-speed sections, while shallower gradients correspond to braking or tighter, slower corners.

Question 2

The raw longitudinal (a_x) and lateral (a_y) acceleration signals showed significant high-frequency noise. This noise is characteristic of raw accelerometer data in a rally environment, likely caused by engine vibration or road surface roughness. Using such noisy data for further calculations would result in unreliable outputs. This setup can be seen in the MATLAB script in Figure 4 in the appendix. The script prepares the noisy data for processing between Lines 4 and 7 of the code. Line 4 ensures the time vector is in a column format, while Lines 6-7 combine the time vector with the raw acceleration data to create the matrix inputs required by the simulation. The filtering simulation is executed on Line 10 using the `sim` command, which runs the external Simulink model named "v_dynamics_filter".

To address this, a software filter was designed using the Analog Filter Design block in Simulink which is shown in Figure 5 in the appendix. The filter type chosen was the Butterworth Low-Pass for its maximally flat passband response, ensuring that the magnitude of the actual vehicle acceleration was not distorted in the passband. The cut-off frequency was set to 10 rad/s as vehicle handling dynamics typically occur at low frequencies. A cut-off of 10 rad/s effectively suppresses the higher-frequency engine and road noise while retaining the true cornering and braking forces. A second-order filter was used as it provides a sufficient roll-off rate to attenuate noise without introducing excessive phase lag, which would otherwise delay the signal and misalign it with the speed data. Following the simulation, Lines 13-15 extract the cleaned signals from the output object. The visualization is generated between Lines 18 and 40. Lines 22 and 33 plot the original raw data in grey to serve as a background reference, while Lines 24 and 35 plot the new filtered signals on top in blue and red.

Figure 6 in the appendix compares the raw and filtered signals. The filtered data successfully tracks the trend of the raw data without the disturbances. Crucially, the peak magnitudes of acceleration are preserved as the filter does not aggressively attenuate the maximum forces experienced during heavy braking or hard cornering. This confirms that the 10 rad/s cut-off frequency was an appropriate choice, cleaning the signal while maintaining data integrity.

Question 3

To quantify the curvature of the rally stage, the path curvature was derived as a function of distance travelled. Unlike yaw rate, which measures rotational velocity, curvature represents the inverse of the instantaneous turning radius ($\frac{1}{R}$) and defines the physical sharpness of the circuit regardless of the vehicle's speed. The calculation relies on isolating the centripetal acceleration acting perpendicular to the vehicle's velocity vector. The computational process is detailed in the MATLAB script in Figure 7 in the

appendix. The script begins by standardizing the input data into SI units between Lines 4 and 15 to ensure mathematical consistency. Line 5 defines the gravitational constant, which is applied in Lines 8–9 to convert the filtered acceleration signals from g-force to $\frac{m}{s^2}$. Simultaneously, Line 12 converts the vehicle speed from km/h to m/s, and Line 15 transforms the sideslip angle from degrees to radians to ensure compatibility with trigonometric functions. Since the onboard accelerometers are fixed to the vehicle's body frame, the filtered longitudinal and lateral acceleration signals from the previous task were projected onto the path coordinate system using the sideslip angle. This transformation is defined by the relationship:

$$a_{centripetal} = a_y \cos(\beta) - a_x \sin(\beta)$$

The final curvature was then calculated as the ratio of centripetal acceleration to the square of the vehicle speed (V^2), with a small constant (ϵ) added to the denominator to prevent numerical instability during instances where the vehicle speed approached zero. To improve the legibility of the plot, a Gaussian smoothing function with a was applied to the raw curvature data.

The resulting curvature profile, plotted against distance in Figure 8 in the appendix, provides a clear geometric map of the rally stage. The results indicate a winding track characterized by frequent, sharp directional changes. Sharp negative spikes, such as the one observed at approximately 1000m indicate a very tight right-hand turn where $|k|$ exceeds 0.6 m^{-1} , corresponding to a turning radius of less than 2m. Conversely, the segments of the graph where $|k|$ remains near zero represent straight sections of the course where the driver can prioritize longitudinal acceleration over cornering. The alternating nature of the peaks confirms a sequence of chicanes and technical corners that demand precise handling and significant lateral grip.

Question 4

The evaluation of driver performance throughout the rally stage was conducted by isolating a full lap of the circuit and determining the local maximum magnitude of curvature at each corner to establish a baseline for theoretical limits. Using the peak detection algorithm shown in the MATLAB script in Figure 9 in the appendix, 16 distinct cornering events were identified where the curvature magnitude exceeded a threshold of 0.04 m^{-1} . The maximum observed lateral acceleration from the filtered dataset was found to be 0.82g which was utilized as the friction limit for the gravel surface. Based on this limit, the theoretical maximum allowable speed (V_{max}) for each specific corner was calculated using the relationship:

$$V_{max} = \sqrt{\frac{a_{max}}{k_{peak}}}$$

As presented in Table 1 in the appendix, a comparison between the driver's actual apex speeds and these theoretical limits reveals varying degrees of grip utilization across the stage. In the most technical sections, the driver operated extremely close to the vehicle's physical limits. For example, at Corner 13, the driver achieved a speed of 11.21km/h against a theoretical maximum of 12.93km/h, representing a high level of performance. Conversely, significant discrepancies were noted at the start of the stage, such as in Corner 1, where the actual speed was only 1.43km/h despite a theoretical limit of 12.36km/h. This high discrepancy is interpreted as the vehicle was launching from a standstill and had not yet reached a velocity high enough to be limited by lateral friction.

The identification of these corners is visually verified in Figure 10 in the appendix, where the red markers align precisely with the curvature peaks throughout the 1380m. Overall, the data suggests that while the driver prioritized exit speed or safety in wider sections, they effectively exploited the available grip throughout the track

Part 2

Question 1

The dynamic behaviour of the DC motor was first analysed in an open-loop configuration to establish a baseline for its natural response characteristics. Using the physical parameters provided in the coursework brief, a second-order transfer function was derived. This mathematical model, shown in Figure 11, relates the input voltage to the output rotational speed.

Upon applying a 1V step input, the open-loop response illustrated in Figure 12 in the appendix shows a smooth, overdamped rise toward a steady-state value. The motor reaches a final speed of approximately 3.92rad/s. This result highlights a significant limitation of the uncontrolled system: the motor possesses no mechanism to reach a specific target speed without manual voltage adjustment. Furthermore, the rise time is relatively slow, taking over one second to settle.

Question 2

To achieve precise speed control, P, PI, and PID controllers were designed and simulated using a unity feedback architecture as detailed in the MATLAB script in Figure

13 in the appendix. Each controller was evaluated based on its ability to track a unit step reference (1rad/s), with specific focus on rise time, settling time, overshoot, and steady-state error. The specific gain values were determined using a step-by-step heuristic tuning process. The proportional gain was first set to $K_p = 5$ to maximize response speed, followed by the introduction of $K_i = 15$ to aggressively eliminate steady-state error. Finally, the derivative term was added to dampen the resulting oscillations, with the initial and tuned values selected based on minimizing the settling time.

The Proportional (P) controller, utilizing a gain of $K_p = 5$, provided a fast initial response with a rise time of 0.068s, but it suffered from a steady-state error of 0.049, as it could not fully overcome the system's internal damping. The Proportional-Integral (PI) controller eliminated this error through the addition of integral action ($K_i = 15$), but this came at the cost of significant instability, resulting in a massive overshoot of 67.9% and a prolonged settling time of 1.87s.

The Proportional-Integral-Derivative (PID) controller provided the most balanced performance by introducing a derivative term ($K_d = 0.1$) to act as a damper. As shown in Figure 14 and Table 2 in the appendix, the PID controller maintained zero steady-state error while reducing the overshoot to 40% and significantly improving the settling time to 0.80s. While this initial PID design was effective, further tuning demonstrated that increasing the derivative gain can even further suppress oscillations, making it the superior choice for applications requiring both accuracy and high-speed stability. To optimize the control system beyond the initial design, the derivative gain (K_d) was increased from 0.1 to 0.4 to provide greater damping against the significant oscillations observed in the aggressive PI and PID responses. Simultaneously, the integral gain (K_i) was slightly reduced to 10 to minimize the phase lag that contributed to the high initial overshoot. These targeted parameter adjustments allowed the system to maintain zero steady-state error while drastically reducing overshoot to 7%, resulting in a much more stable and professional motor response. The PID controller also settled within 0.28 seconds showing improved damping.

Appendix

```

Part1Question1.m x Part1Question2.m x Part1Question3.m x +
MATLAB DrivePart1Question1.m
1 % PART 1, TASK 1: YAW ANGLE & DISTANCE CALCULATION
2
3 clear all; clc; close all;
4
5 load('data_1.mat');
6
7 % 3. PRE-PROCESS DATA FOR SIMULINK
8 % a. Force time vector to column
9 time_col = t(:);
10
11 % b. Prepare Yaw Rate (deg/s)
12 yaw_rate_col = psidot(:);
13 yaw_rate_input = [time_col, yaw_rate_col];
14
15 % c. Prepare Speed (km/h -> m/s)
16 V_ms = V(:) / 3.6;
17 speed_input = [time_col, V_ms];
18
19 % d. RUN SIMULINK MODEL
20 simOut = sim('v_dynamics_model');
21
22 % 5. EXTRACT RESULTS
23 t_sim = simOut.tout;
24 yaw_angle_deg = simOut.yaw_angle_sim;
25 distance_m = simOut.distance_sim;
26
27 % 6. GENERATE PLOT
28 figure('Name', 'Part 1-Task 1: Yaw and Distance', 'color', 'w');
29
30 % Subplot 1: Yaw Angle
31 subplot(2,1,1);
32 plot(t_sim, yaw_angle_deg, 'b', 'linewidth', 1.5);
33 grid on;
34 title('Vehicle Yaw Angle vs Time', 'FontSize', 12, 'FontWeight', 'bold');
35 xlabel('Time (s)');
36 ylabel('Yaw Angle (deg)');
37 legend('Yaw Angle (\psi)', 'location', 'best');
38
39 % Subplot 2: Distance Travelled
40 subplot(2,1,2);
41 plot(t_sim, distance_m, 'r', 'linewidth', 1.5);
42 grid on;
43 title('Distance Travelled vs Time', 'FontSize', 12, 'FontWeight', 'bold');
44 xlabel('Time (s)');
45 ylabel('Distance (m)');
46 legend('Distance (s)', 'location', 'best');

```

Figure 1: MATLAB script used to pre-process raw data and extract simulation outputs for yaw angle and distance calculations.

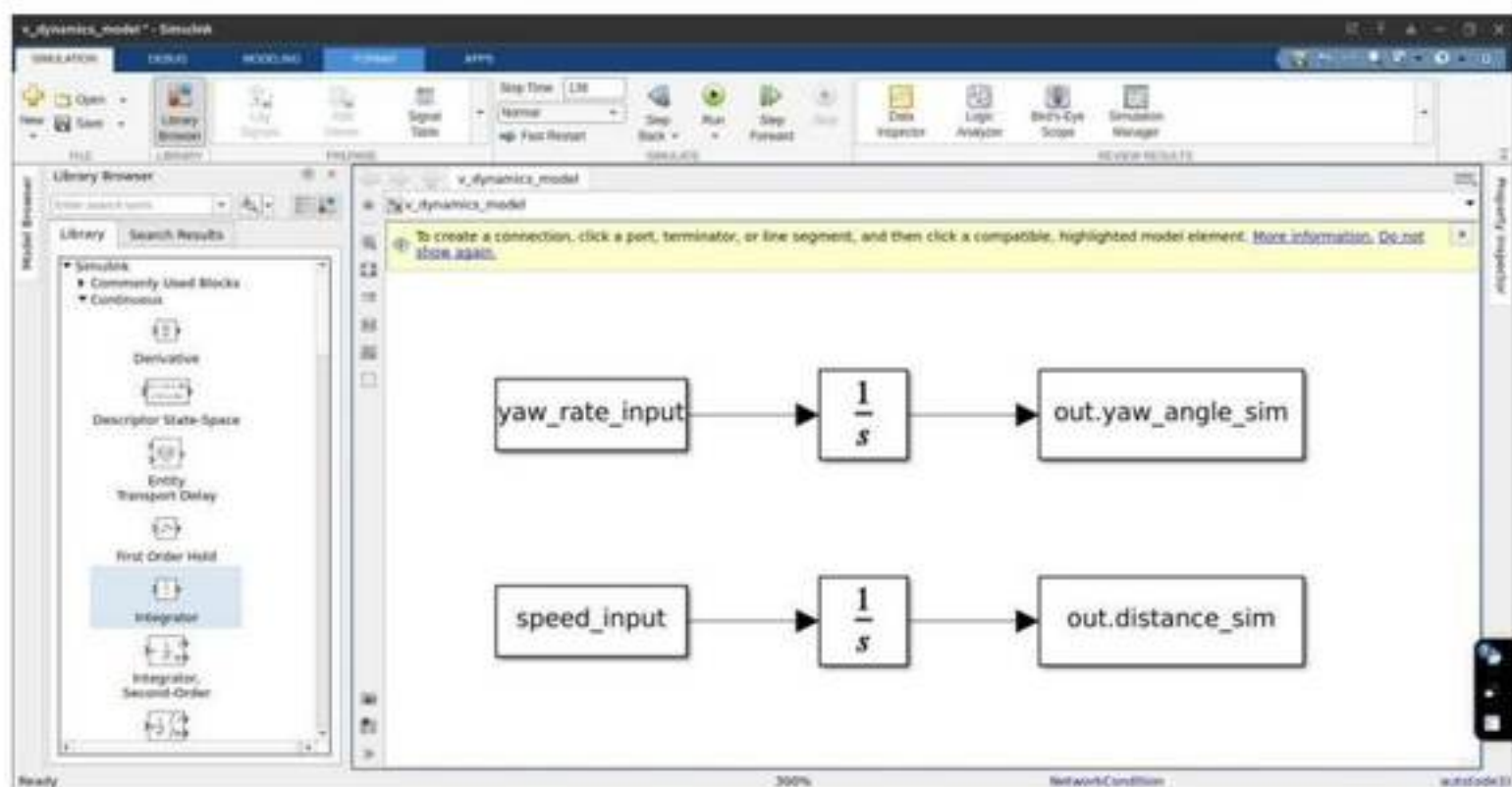


Figure 2: Simulink model configured with a Fixed-Step solver (0.01s) to numerically integrate yaw rate and velocity signals.

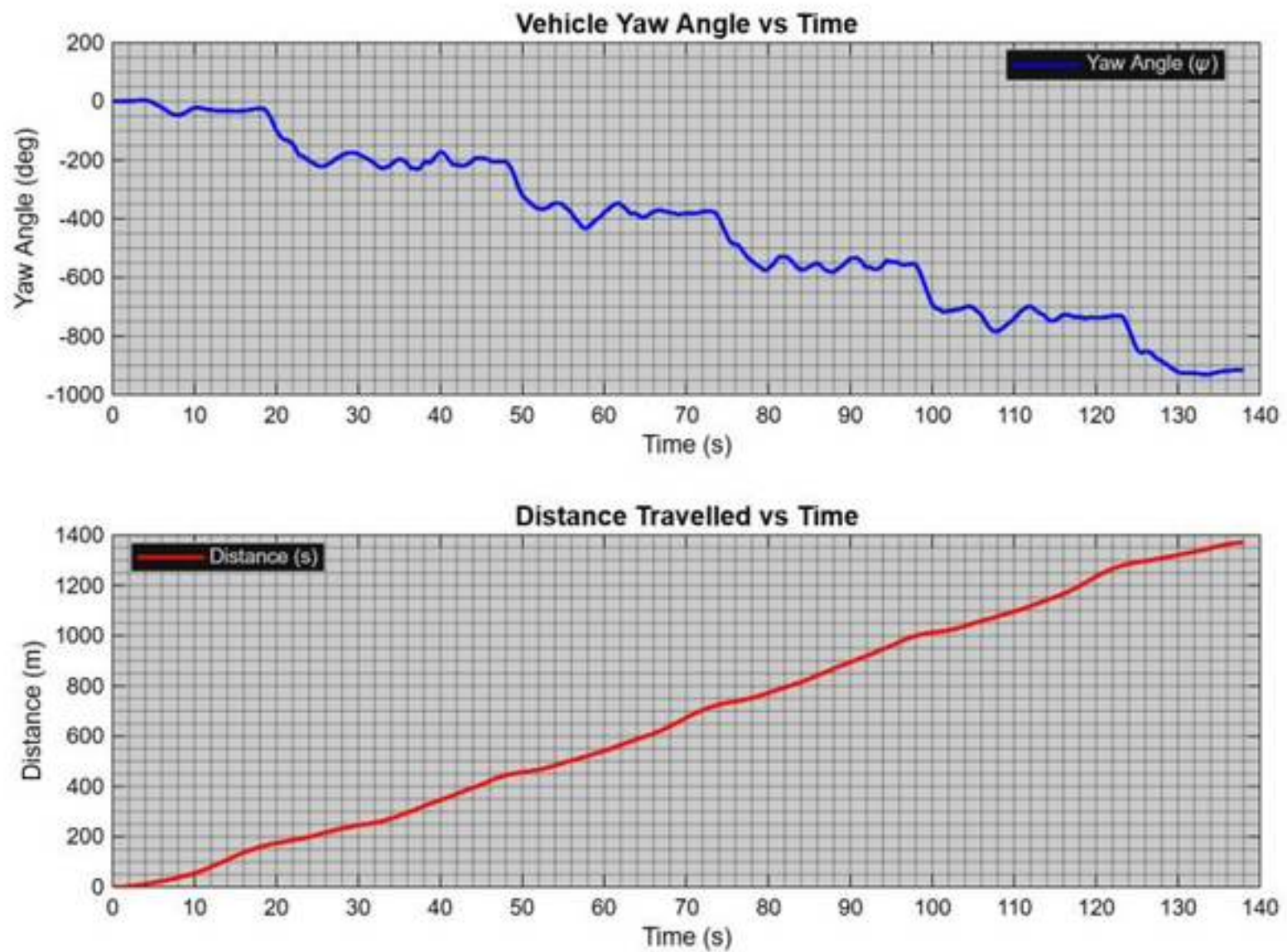


Figure 3: Calculated Vehicle Yaw Angle and Cumulative Distance Travelled vs. Time, derived from the integration of raw sensor data.

```

MATLAB DrivePart1Question2.m
1  % PART 1, TASK 2: ACCELERATION FILTERING
2
3  % 2. PRE-PROCESS DATA
4  % force inputs into columns
5  time_col = t(:);
6  ax_input = [time_col, ax(:)];
7  ay_input = [time_col, ay(:)];
8
9  % 3. RUN SIMULINK MODEL
10 simOut = sim('v_dynamics_filter');
11
12 % 4. EXTRACT RESULTS
13 ax_smooth = simOut.ax_filtered;
14 ay_smooth = simOut.ay_filtered;
15 t_sim = simOut.tout;
16
17 % 5. GENERATE COMPARISON PLOTS
18 figure('Name', 'Part 1 Task 2: Filtering Analysis', 'Color', 'w');
19
20 % Subplot 1: Longitudinal Acceleration (ax)
21 subplot(2,1,1);
22 plot(t, ax, 'Color', [0.7 0.7 0.7]); % Raw data in Grey
23 hold on;
24 plot(t_sim, ax_smooth, 'b', 'linewidth', 1.5); % Filtered in Blue
25 grid on;
26 title('Longitudinal Acceleration (ax): Raw vs Filtered', 'FontSize', 12, 'FontWeight', 'bold');
27 ylabel('Acceleration (g)');
28 xlabel('Time (s)');
29 legend('Raw Signal', 'Filtered Signal (Butterworth 10 rad/s)');
30
31 % Subplot 2: Lateral Acceleration (ay)
32 subplot(2,1,2);
33 plot(t, ay, 'Color', [0.7 0.7 0.7]); % Raw data in Grey
34 hold on;
35 plot(t_sim, ay_smooth, 'r', 'linewidth', 1.5); % Filtered in Red
36 grid on;
37 title('Lateral Acceleration (ay): Raw vs Filtered', 'FontSize', 12, 'FontWeight', 'bold');
38 ylabel('Acceleration (g)');
39 xlabel('Time (s)');
40 legend('Raw Signal', 'Filtered Signal (Butterworth 10 rad/s)');

```

Figure 4: MATLAB script for pre-processing acceleration data and running the filter simulation

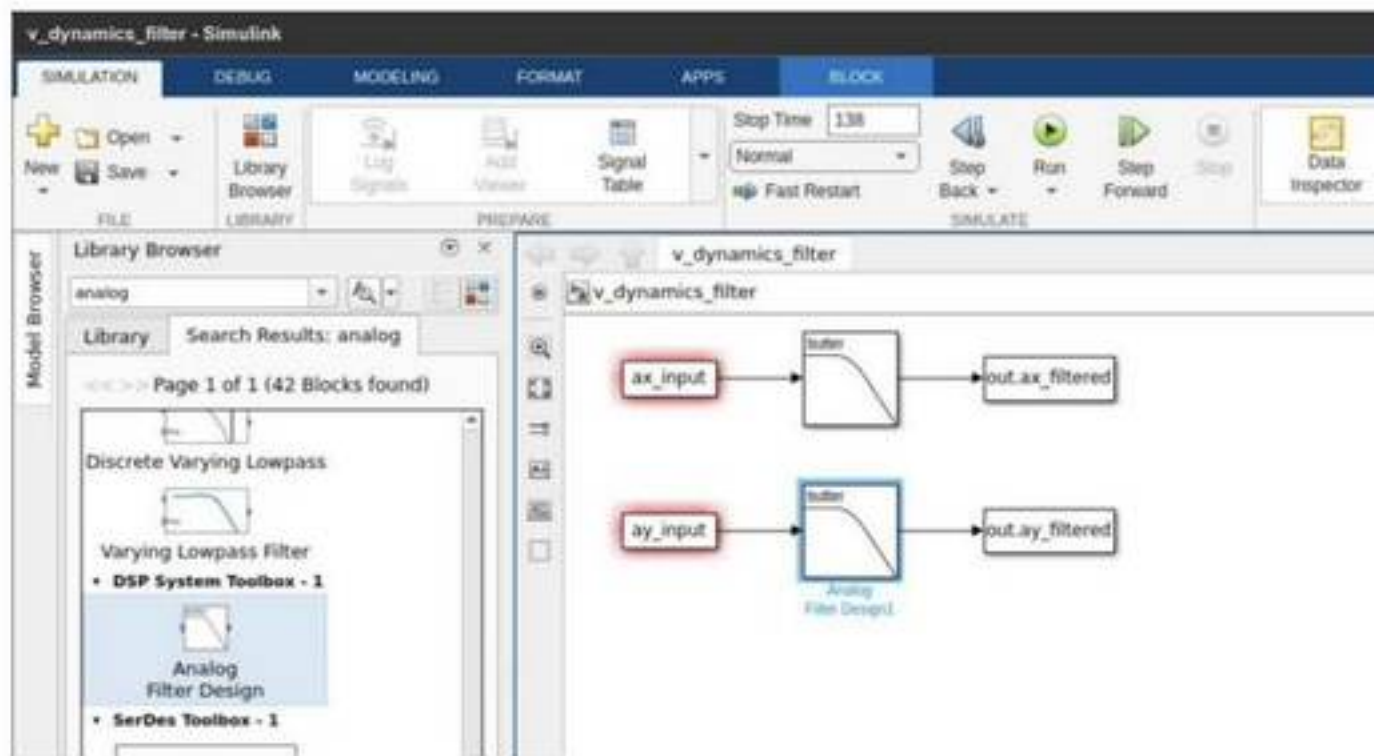


Figure 5: Simulink filter design model implementing a 2nd-order Butterworth Low-Pass filter with a cut-off frequency of 10 rad/s

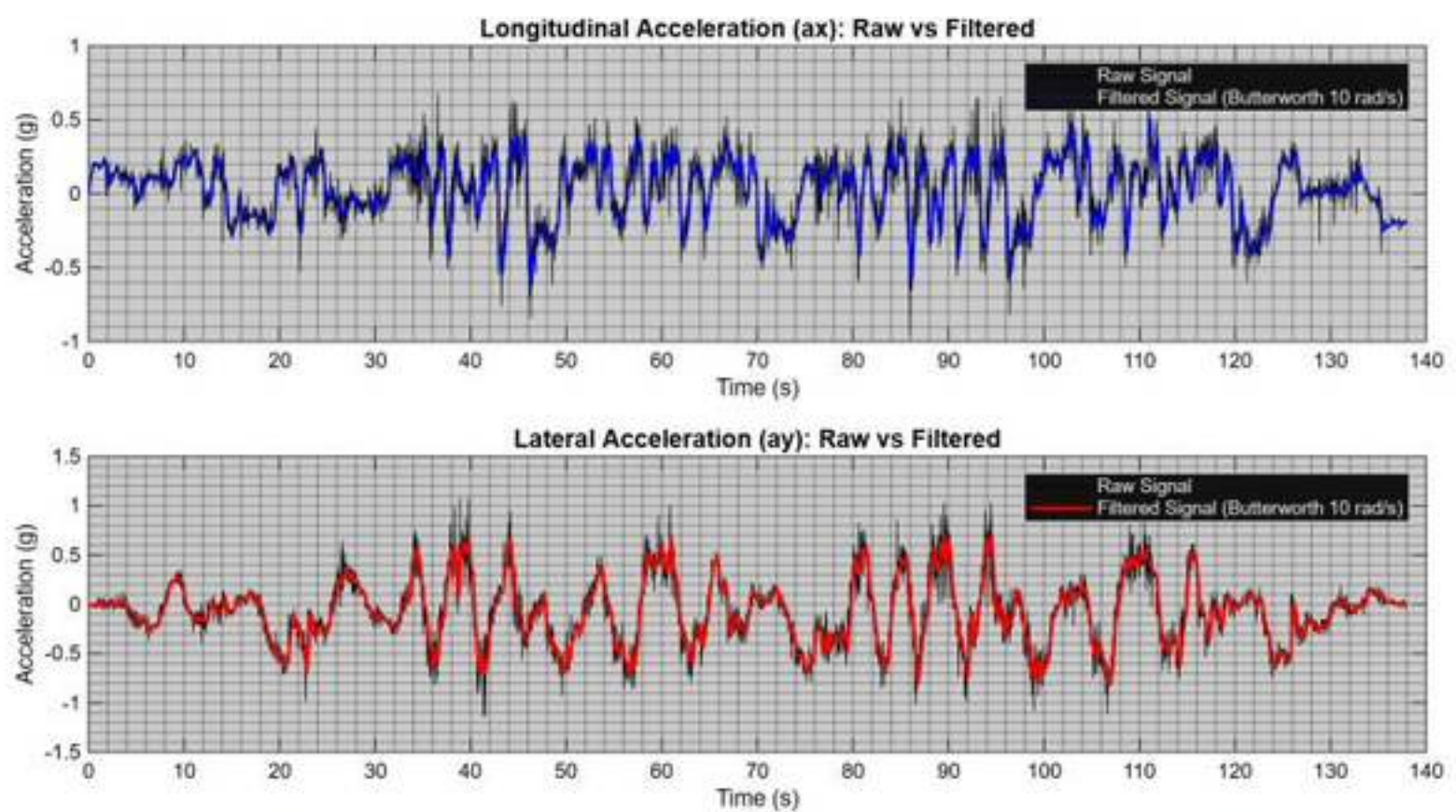


Figure 6: Comparison of Raw (Grey) vs. Filtered (Blue/Red) Acceleration signals. The filter effectively suppresses high-frequency noise while preserving dynamic peaks.

```

Part1Question1.m X Part1Question2.m X Part1Question3.m X +
/MATLAB Drive/Part1Question3.m
1  %% PART 1, TASK 3: PATH CURVATURE CALCULATION
2
3
4  % 1. UNIT CONVERSIONS
5  g = 9.81;
6
7  % Convert Filtered Accelerations from 'g' to 'm/s^2'
8  ax_ms2 = ax_smooth * g;
9  ay_ms2 = ay_smooth * g;
10
11 % Convert Speed from km/h to m/s
12 V_ms = V(:) / 3.6;
13
14 % Convert Sideslip from degrees to radians.
15 beta_rad = deg2rad(beta(:));
16
17 % 2. CALCULATE CENTRIPETAL ACCELERATION
18 a_cent = ay_ms2 .* cos(beta_rad) - ax_ms2 .* sin(beta_rad);
19
20 % 3. CALCULATE CURVATURE (k)
21 % Formula: k = a_cent / V^2
22 k_raw = a_cent ./ (V_ms.^2 + eps);
23
24 % 4. POST-PROCESSING
25 k_final = smoothdata(k_raw, 'gaussian', 20);
26
27 % 5. PLOT RESULTS
28 figure('Name', 'Part 1 Task 3: Path Curvature', 'Color', 'w');
29
30 plot(distance_m, k_final, 'b', 'LineWidth', 1.2);
31 grid on;
32 title('Path Curvature vs Distance Travelled', 'FontSize', 12, 'FontWeight', 'bold');
33 xlabel('Distance Travelled (m)');
34 ylabel('Curvature k (1/m)');
35 legend('Path Curvature (k)');
36 ylim([-0.7 0.2]);

```

Figure 7: MATLAB script for calculating path curvature using vehicle speed, sideslip angle, and filtered body accelerations



Figure 8: Path Curvature profile plotted against Distance Travelled, showing the nature of the rally stage.


```

1  %% PART 1, TASK 4: CORNERING ANALYSIS
2
3  % 2. DETERMINE MAXIMUM GRIP (a_max)
4  a_max_g = max(abs(ay_smooth));
5  a_max_ms2 = a_max_g * 9.81; % Convert to m/s^2
6
7  % 3. IDENTIFY CORNERS (PEAK FINDING)
8  [k_peaks, locs] = findpeaks(abs(k_final), 'MinPeakHeight', 0.04, 'MinPeakDistance', 500);
9
10 % 4. EXTRACT DATA AT CORNERS
11 corner_dist = distance_m(locs);
12 v_actual_ms = V_ms(locs);
13
14 % 5. CALCULATE THEORETICAL MAX SPEED
15 v_theory_ms = sqrt(a_max_ms2 ./ k_peaks);
16
17 % 6. GENERATE COMPARISON TABLE
18 results_table = table((1:length(locs))', corner_dist, k_peaks, v_actual_ms*3.6, v_theory_ms*3.6, ...
19     'VariableNames', {'Corner_ID', 'Distance_m', 'Peak_Curvature_k', 'Actual_Speed_kmh', 'Theoretical_Max_kmh'});
20
21 disp('Cornering Analysis Results:');
22 disp(results_table);
23
24 % 7. PLOT FOR VERIFICATION
25 figure('Name', 'Part 1 Task 4: Identified Corners', 'Color', 'w');
26 plot(distance_m, abs(k_final), 'b'); hold on;
27 plot(corner_dist, k_peaks, 'ro', 'MarkerSize', 6, 'LineWidth', 2);
28 grid on;
29 title('Identified Corners on Rally Stage', 'FontSize', 12, 'FontWeight', 'bold');
30 xlabel('Distance Travelled (m)');
31 ylabel('Curvature Magnitude |k| (1/m)');
32 legend('Path Curvature', 'Identified Corners');

```

Figure 9: MATLAB script performing peak detection to identify corners and calculate theoretical maximum cornering speeds

Table 1: Cornering Analysis Results comparing the driver's Actual Apex Speed against the Theoretical Maximum Speed based on a lateral grip limit of 0.82g.

CORNER ID	DISTANCE (M)	PEAK CURVATURE K (1/M)	ACTUAL SPEED (KM/H)	THEORETICAL MAX SPEED (KM/H)
1	0.04	0.685	1.43	12.36
2	26.32	0.062	23.17	41.20
3	179.15	0.212	19.61	22.23
4	252.51	0.064	18.29	40.45
5	326.19	0.043	42.20	49.54
6	396.02	0.053	39.77	44.52
7	458.75	0.314	14.81	18.23
8	507.93	0.092	30.70	33.70
9	578.67	0.052	38.56	44.97
10	734.88	0.208	20.39	22.41
11	809.04	0.062	36.24	41.10
12	918.98	0.050	43.73	45.76
13	1013.90	0.626	11.21	12.93
14	1065.60	0.116	29.81	30.07
15	1144.20	0.046	41.48	47.92
16	1294.60	0.319	15.13	18.09

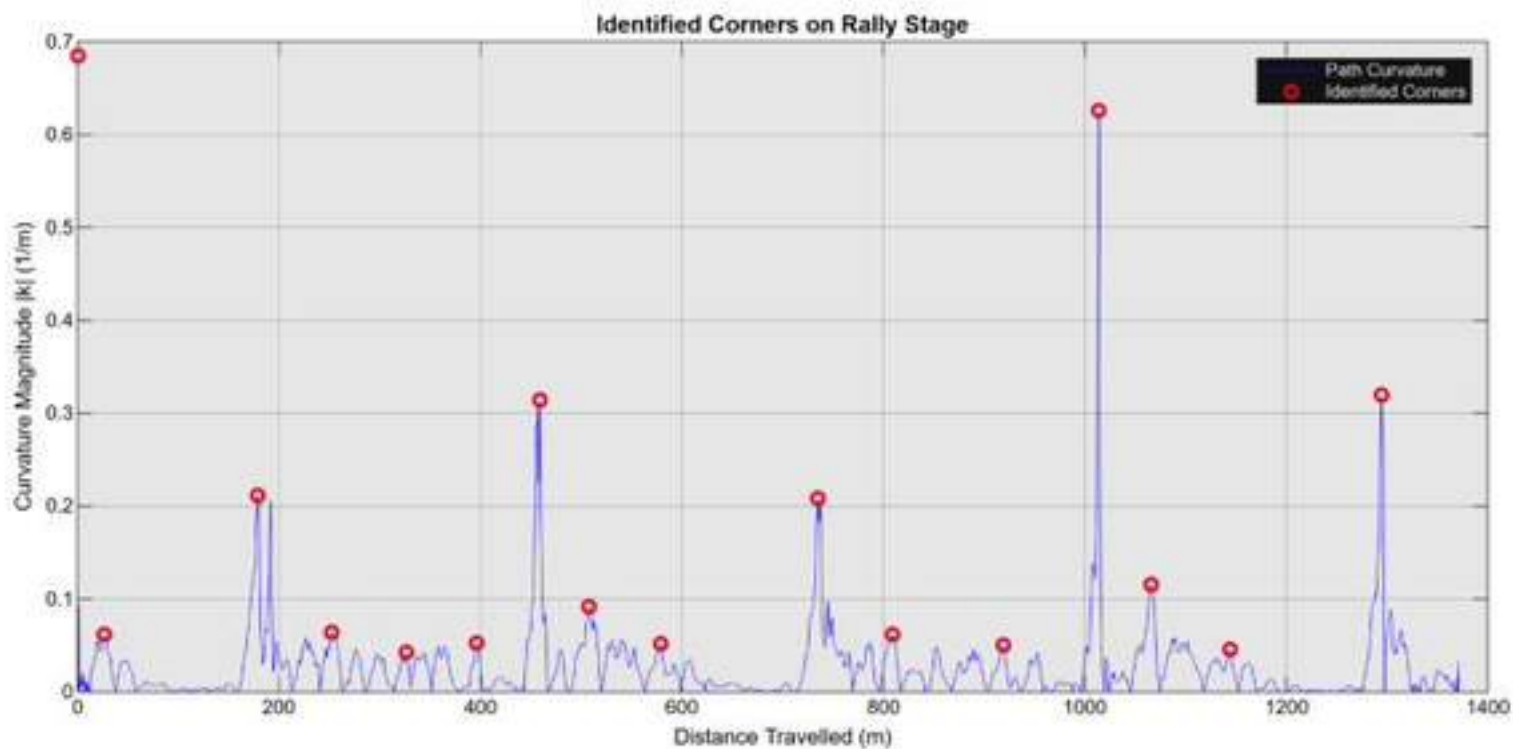


Figure 10: Corner identification results showing local maxima in curvature magnitude used to isolate specific cornering events.

```

Part1Question1.m X Part1Question2.m X Part1Question3.m X Part1Question4.m X
MATLAB DrivePartQ1.m
1 % XX PART 2, TASK 1: DC MOTOR OPEN LOOP RESPONSE
2 % Requirement: Plot the step response using the transfer function.
3 %
4
5 clear all; clc; close all;
6
7 % 1. DEFINE SYSTEM PARAMETERS
8 % Values taken directly from the coursework brief
9 J = 0.0113; % Moment of Inertia (kg*m^2)
10 b = 0.028; % Viscous Damping (Nms)
11 L = 0.1; % Armature Inductance (H)
12 R = 0.45; % Armature Resistance (Ohm)
13 Kt = 0.067; % Torque Constant (Nm/Amp)
14 Ke = 0.067; % Back EMF Constant (Vs/rad)
15
16 % 2. CREATE TRANSFER FUNCTION
17 % Formula: G(s) = Kt / [ (Ls+R)*(Js+b) + Kt*Ke ]
18 % We represent 's' as the Laplace variable
19 s = tf('s');
20
21 % Define the numerator and denominator
22 numerator = Kt;
23 denominator = (L*s + R) * (J*s + b) + (Kt * Ke);
24
25 % Create the system
26 G_motor = numerator / denominator;
27
28 % Display the Transfer Function in the Command Window
29 disp('DC Motor Transfer Function:');
30 G_motor
31
32 % 3. PLOT STEP RESPONSE
33 % This shows what happens if we apply a 1V step input
34 figure('Name', 'Part 2 Task 1: DC Motor Step Response', 'Color', 'w');
35 step(G_motor);
36 grid on;
37 title('Open Loop Step Response of DC Motor', 'Color', 'k');
38 ylabel('Speed (rad/s)', 'Color', 'k');
39 xlabel('Time (s)', 'Color', 'k');
40
41 % 4. EXTRACT CHARACTERISTICS
42 % We can get the steady-state speed automatically
43 info = stepinfo(G_motor);
44 fprintf('Steady State Speed for 1V input: %.4f rad/s\n', info.SettlingMin);

```

Figure 11: MATLAB script defining DC motor parameters and generating the open-loop transfer function.

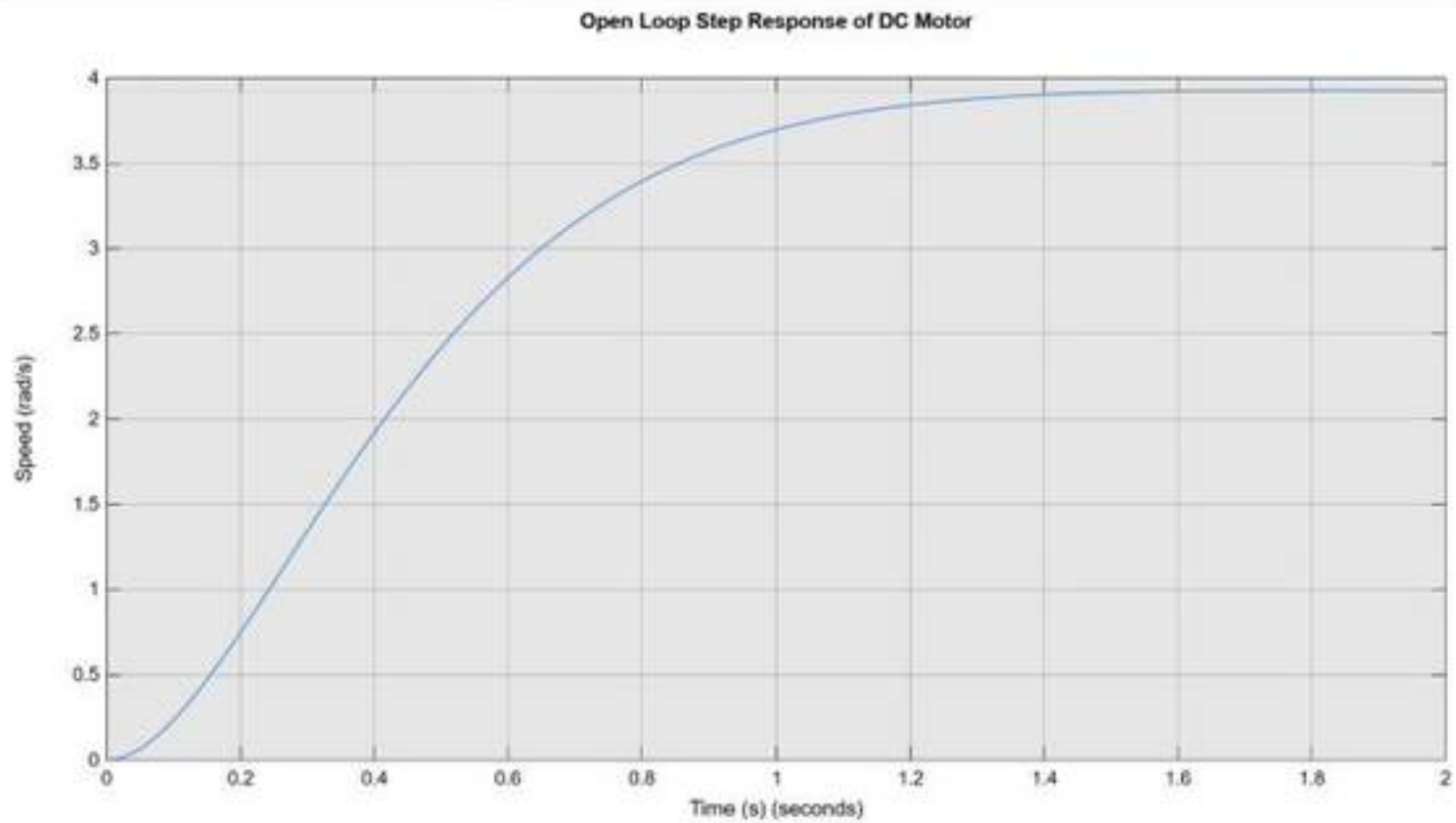


Figure 12: Open-Loop Step Response of the DC motor to a 1V input, showing a steady-state settling speed of approximately 3.92 rad/s.

```

Part1Question1.m X Part1Question2.m X Part1Question3.m X Part1Question4.m X Part2Q1
MATLAB (OneParQ12).m
1 % PART 2, TASK 2: P, PI, PID CONTROLLER DESIGN & COMPARISON
2 clear all; clc; close all;
3
4 % 1. DEFINE DC MOTOR
5 J = 0.0113; b = 0.028; L = 0.1; R = 0.45; Kt = 0.067; Ke = 0.067;
6 s = tf('s');
7 G_motor = Kt / ( (L*s + R)*(J*s + b) + Kt*Ke );
8
9 % 2. DESIGN CONTROLLERS
10
11 % P Controller
12 Kp_p = 5;
13 C_p = pid(Kp_p, 0, 0);
14
15 % PI controller
16 K
17 Kp_pi = 5; Ki_pi = 15;
18 C_pi = pid(Kp_pi, Ki_pi, 0);
19
20 % PID Controller
21 Kp_pid = 5; Ki_pid = 15; Kd_pid = 0.1;
22 C_pid = pid(Kp_pid, Ki_pid, Kd_pid);
23
24 % 3. CREATE FEEDBACK LOOPS
25 Sys_P = feedback(C_p * G_motor, 1);
26 Sys_PI = feedback(C_pi * G_motor, 1);
27 Sys_PID = feedback(C_pid * G_motor, 1);
28
29 % 4. PLOT COMPARISON
30 figure('Name', 'Part 2 Task 2: Controller Comparison', 'Color', 'w');
31 t = 0:0.001:3; % Simulate for 3 seconds
32 step(Sys_P, 'b', Sys_PI, 'r--', Sys_PID, 'g', t);
33 grid on;
34 legend('P Only (Kp=5)', 'PI (Kp=5, Ki=15)', 'PID (Kp=5, Ki=15, Kd=0.1)', 'location', 'southeast');
35 title('Step Response Comparison of Controllers', 'Color', 'k');
36 ylabel('Speed (rad/s)', 'color', 'k');
37 xlabel('Time (s)', 'color', 'k');
38
39 % 5. CALCULATE METRICS FOR TABLE
40 get_metrics = @(sys) [ ...
41     stepinfo(sys).RiseTime; ...
42     stepinfo(sys).SettlingTime; ...
43     stepinfo(sys).Overshoot; ...
44     1.0 - dcgain(sys)]; % Error = Target - Actual
45
46 % compile results
47 P_res = get_metrics(Sys_P);
48 PI_res = get_metrics(Sys_PI);
49 PID_res = get_metrics(Sys_PID);
50
51 % Create Table
52 Metric_Names = {'Rise Time (s)'; 'Settling Time (s)'; 'Overshoot (%)'; 'Steady State Error'};
53 results_table = table(P_res, PI_res, PID_res, ...
54     'RowNames', Metric_Names, ...
55     'VariableNames', {'P_Controller', 'PI_Controller', 'PID_Controller'});
56

```

Figure 13: MATLAB script designing P, PI, and PID control architectures and computing performance metrics.

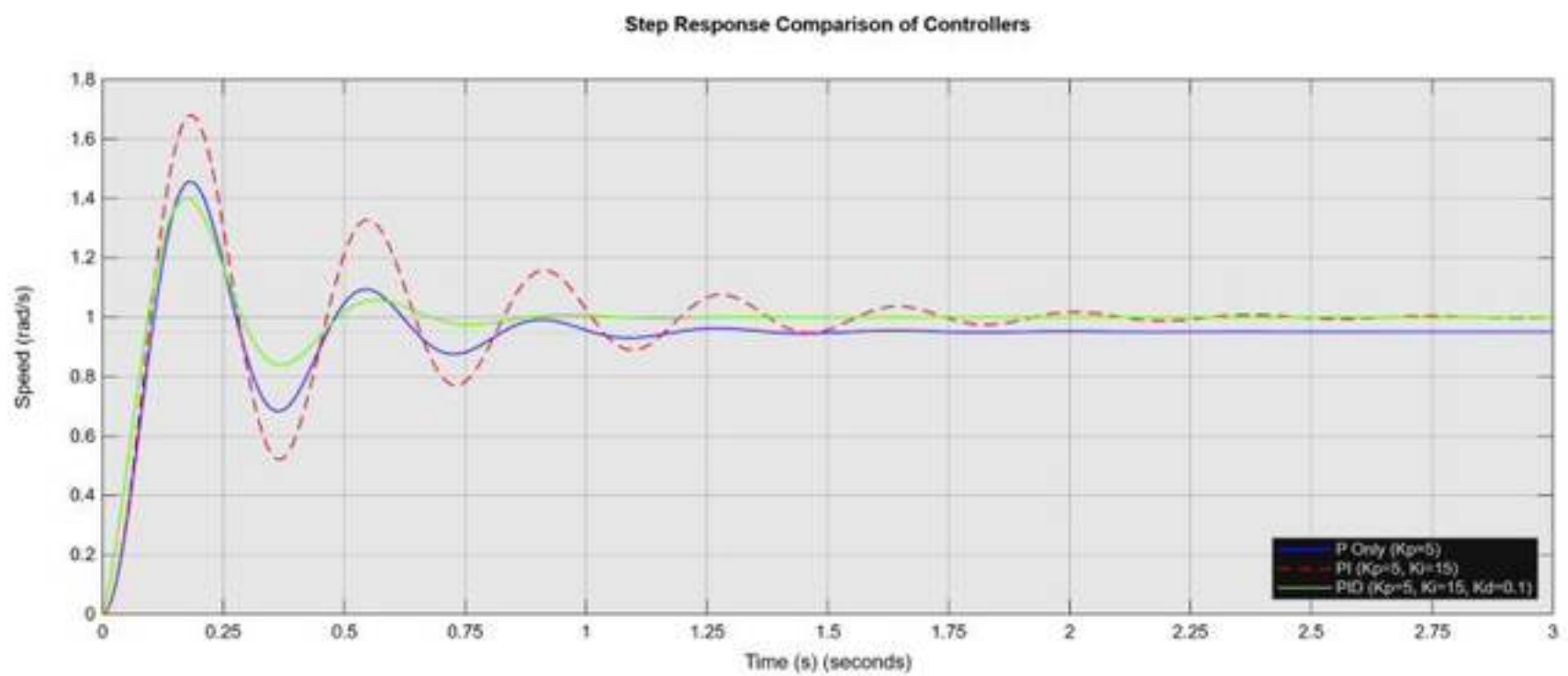


Figure 14: Closed-Loop Step Response comparison of P, PI, and PID controllers tracking a unit step reference.

Table 2: Performance Comparison Metrics for P, PI, and PID controllers, highlighting the trade-offs between Rise Time, Settling Time, Overshoot, and Steady-State Error.

METRIC	P CONTROLLER	PI CONTROLLER	PID CONTROLLER
GAINS USED	$K_p = 5$	$K_p = 5, K_i = 15$	$K_p = 5, K_i = 15, K_d = 0.1$
RISE TIME (S)	0.068	0.066	0.072
SETTLING TIME (S)	1.12	1.87	0.80
OVERSHOOT (%)	53.0%	67.9%	40.0%
STEADY STATE ERROR	0.049	0	0

Table 3: Performance Comparison Metrics for the original PID and the optimised PID controllers, highlighting the trade-offs between Rise Time, Settling Time, Overshoot, and Steady-State Error.

METRIC	ORIGINAL PID	TUNED PID
GAINS USED	$K_p = 5, K_i = 15, K_d = 0.1$	$K_p = 5, K_i = 10, K_d = 0.4$
RISE TIME (S)	0.072	0.065
SETTLING TIME (S)	0.8	0.28
OVERSHOOT (%)	40.0%	7%
STEADY STATE ERROR	0	0